

Diseño Arquitectónico Integral de Base de Datos y Capa Transaccional API para Plataforma SaaS Logística Offline-First

Contexto Arquitectónico y Fundamentos del Sistema Distribuido

La conceptualización de un sistema integral de gestión para una panificadora con distribución en furgonetas requiere una arquitectura de software que concilie la necesidad de un control administrativo centralizado en tiempo real con la realidad física de una conectividad intermitente en las rutas de reparto. El desafío central radica en construir una base de datos robusta en Supabase (PostgreSQL) que actúe como la única fuente de verdad, al tiempo que se sincroniza bidireccionalmente con bases de datos SQLite integradas en los dispositivos móviles de los repartidores mediante un motor de sincronización de datos local como PowerSync.¹

El ecosistema operativo se divide en dos grandes interfaces con requerimientos técnicos opuestos. Por un lado, el panel web administrativo exige visualizaciones complejas, métricas globales, auditorías cruzadas de stock y gestión de cuentas corrientes, demandando consultas de agregación masivas y uniones de tablas (JOINS) eficientes.⁴ Por otro lado, la aplicación móvil para repartidores debe garantizar una operatividad sin latencia, permitiendo registrar ventas, devoluciones y cobros con una interfaz diseñada para usarse con una sola mano y botones de acceso rápido, incluso en zonas rurales sin señal celular.⁴ Para lograr esta operatividad ininterrumpida, el diseño de la base de datos no puede depender de las llamadas de red tradicionales (API REST estándar), sino que debe basarse en un modelo de sincronización parcial, donde las lecturas y escrituras móviles ocurren contra una réplica local que, mediante políticas de resolución causal, converge con el servidor maestro al recuperar la conectividad.¹ Este documento detalla la estructura relacional exhaustiva, la capa de Interfaz de Programación de Aplicaciones (API) basada en Procedimientos Almacenados (RPC), las configuraciones a nivel de clúster, y las políticas de seguridad a nivel de fila (Row Level Security - RLS) diseñadas para evitar la degradación del rendimiento en un entorno distribuido de alta concurrencia.

Configuraciones Críticas a Nivel de Clúster

PostgreSQL

El diseño de un sistema transaccional distribuido exige establecer parámetros a nivel de servidor que garanticen la integridad temporal y la persistencia lógica de los datos. Las configuraciones por defecto de un clúster de Supabase requieren modificaciones específicas para adaptarse a la operativa diaria de una empresa de logística en Sudamérica.

La gestión del tiempo es un pilar crítico en aplicaciones de reparto sincronizadas. Todos los campos temporales del sistema deben utilizar ineludiblemente el tipo de dato `timestampz`

(Timestamp with Time Zone), el cual almacena internamente el valor en Tiempo Universal Coordinado (UTC) pero permite conversiones dinámicas automáticas según el contexto de la consulta.⁷ Para la plataforma logística en cuestión, el servidor PostgreSQL debe configurarse globalmente para operar bajo el huso horario America/Argentina/Buenos_Aires.⁹ Esta configuración administrativa garantiza que las consultas de agregación diaria, como el corte de caja del panel administrativo o el resumen de métricas globales, reflejen con exactitud el desfase de UTC-3.¹¹ La ausencia de esta configuración provocaría anomalías críticas donde las transacciones realizadas a última hora de la tarde se registrarían contablemente en la jornada del día siguiente, destruyendo la fiabilidad del cruce automático de stock post-ensado y ventas.¹² Para asegurar la coherencia en todos los entornos, la variable timezone en el archivo de configuración de PostgreSQL o mediante el comando ALTER DATABASE debe fijarse estrictamente a este huso horario.¹⁰

En conjunción con la gestión temporal, el paradigma offline-first prohíbe categóricamente la eliminación física de registros mediante comandos DELETE estándar en tablas transaccionales o de catálogo. Si un producto se elimina físicamente en el servidor maestro, los clientes móviles desconectados no recibirían una notificación explícita de esta eliminación, lo que resultaría en datos huérfanos, cálculos de inventario inconsistentes y excepciones en la interfaz del usuario al reconectarse.² La arquitectura resuelve este problema implementando un patrón de diseño basado en "tombstones" (lápidas) o eliminación lógica.¹⁴ Cada tabla del esquema relacional incluye una columna is_deleted (booleano). Cuando se descataloga un producto o se anula una factura, el registro simplemente actualiza su estado, permitiendo que el motor de sincronización reconozca el cambio de estado y propague la ocultación del registro a los clientes SQLite locales.¹⁵ Para evitar que el almacenamiento crezca indefinidamente a lo largo de los años, un proceso en segundo plano (vacuum o cron job basado en la extensión pg_cron) se programa para depurar físicamente los tombstones más antiguos de 30 días, garantizando que los clientes inactivos por periodos prolongados requieran una sincronización completa para auto-sanarse, mientras que los clientes activos mantienen un rendimiento óptimo.¹⁴

Dominio de Catálogo de Productos y Control de Producción

El catálogo centraliza los productos manufacturados disponibles para la comercialización. La complejidad arquitectónica de este módulo radica en la multiplicidad de unidades de medida y la obligatoriedad de un sistema de precios diferenciados dinámico.⁴

El diseño relacional de la tabla de productos contempla las unidades estandarizadas del negocio panadero, estableciendo un mapeo directo entre la base de datos y el comportamiento de la interfaz de usuario móvil.⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria. Generado

		por defecto mediante la extensión uuid-osp o pgcrypto con uuid_generate_v4().
name	VARCHAR(150)	No nulo. Indexado con índice B-Tree para búsquedas alfabéticas eficientes en el panel web.
unit_type	VARCHAR(20)	No nulo. Protegido por restricción CHECK (unit_type IN ('kg', 'docena', 'bolsa', 'unidad', 'caja')).
price_a	NUMERIC(10,2)	No nulo. Restricción CHECK (price_a >= 0). Valor comercial para clientes de categoría élite.
price_b	NUMERIC(10,2)	No nulo. Restricción CHECK (price_b >= 0). Valor comercial para consumidores estándar o finales.
bakery_stock	NUMERIC(10,2)	No nulo, valor por defecto 0. Refleja el control cruzado de la producción post-ensado en la central.
is_deleted	BOOLEAN	No nulo, valor por defecto false. Bandera de tombstone para la sincronización offline.
updated_at	TIMESTAMPTZ	No nulo. Actualizado ininterrumpidamente mediante un disparador

		(trigger) para notificar a PowerSync.
--	--	---------------------------------------

El uso inexcusable del tipo NUMERIC(10,2) en lugar de tipos de coma flotante tradicionales (FLOAT o REAL) obedece a un estándar inviolable en la ingeniería de sistemas financieros. El tipo numérico garantiza la precisión aritmética exacta requerida para evitar variaciones por redondeo binario IEEE 754, un factor crítico considerando que la plataforma procesa productos pesables en kilogramos que requieren precisión decimal.⁴

La restricción de la columna unit_type trasciende la simple integridad referencial de la base de datos; actúa como un parámetro de configuración para la Experiencia de Usuario (UX) en la terminal de despacho móvil. Cuando el repartidor selecciona un producto en la interfaz de venta, el frontend evalúa esta columna: si el tipo es kg, la aplicación invoca un teclado numérico con soporte nativo para separadores decimales; inversamente, para unidades como bolsas, docenas y unidades, el sistema operativo móvil restringe la entrada a números enteros, mitigando drásticamente el error de carga rápida de datos en origen.⁴

Para cimentar la comprensión del esquema, el análisis del estado inicial del sistema revela el poblamiento base del catálogo, el cual la base de datos debe ser capaz de soportar sin fricciones.⁴

ID (UUID simplificado)	Nombre del Producto	Unidad	Precio Cat A	Precio Cat B	Stock Panadería Inicial
1	Pan Francés	kg	\$1500.00	\$1300.00	200.00
2	Facturas Surtidas	docena	\$4000.00	\$3600.00	50.00
3	Pan de Miga	bolsa	\$6500.00	\$6000.00	30.00
4	Prepizzas	unidad	\$900.00	\$800.00	40.00
5	Bizcochitos	kg	\$3000.00	\$2800.00	20.00
6	Factura Individual	unidad	\$400.00	\$350.00	150.00

Entidades Operativas: Perfiles de Repartidores y Logística Vehicular

La arquitectura de seguridad se sustenta en la tabla nativa auth.users gestionada por el núcleo de Supabase.¹⁸ Sin embargo, los metadatos operativos, métricas de rendimiento y geolocalización requieren una abstracción en el esquema público que mantenga una integridad referencial estricta, permitiendo así la auditoría desde el panel de control administrativo sin comprometer los secretos de autenticación.

La tabla de perfiles de repartidores se configura como el eje central logístico, sirviendo como pivote para todas las consultas transaccionales vinculadas al rendimiento territorial.⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria.
user_id	UUID	Único. Llave foránea referenciando directamente a auth.users(id).
full_name	VARCHAR(150)	No nulo. Nombre completo del empleado de distribución.
status	VARCHAR(50)	No nulo, defecto 'En Base'. CHECK (status IN ('En Base', 'En Ruta', 'Finalizado')).
is_online	BOOLEAN	No nulo, defecto false. Indicador biométrico de conectividad sincronizado vía sockets.
cash_collected	NUMERIC(12,2)	No nulo, defecto 0. Rendimiento acumulado de efectivo en la jornada actual.

transfer_collected	NUMERIC(12,2)	No nulo, defecto 0. Rendimiento acumulado de pagos digitales en la jornada actual.
location_data	JSONB	Opcional. Almacena latitud, longitud y precisión de la última ubicación conocida.
last_active	TIMESTAMPTZ	No nulo. Actualizado dinámicamente.

El campo estructurado location_data utiliza el formato binario JSONB para permitir futuras integraciones con sistemas de Información Geográfica (GIS) sin alterar el esquema relacional rígido. El estado inicial operativo del sistema refleja la eficacia de esta estructura: repartidores como "Roberto Sánchez" (Estado: En Base, Conectado: Falso, Ventas: \$0) y "Carlos Ruiz" (Estado: Finalizado, Conectado: Falso, Efectivo: \$80,000, Transferencias: \$40,000) son renderizados instantáneamente en el panel web administrativo para evaluar el flujo de caja diario.⁴

Control de Inventario Vehicular y Asignación de Cargas

El éxito de una interfaz offline-first descansa en la viabilidad algorítmica de la sincronización parcial. Intentar replicar toda la base de datos de la empresa en un dispositivo móvil con recursos limitados resulta en tiempos de carga prohibitivos y bloqueos de interfaz.¹⁹ En consecuencia, la base de datos debe estructurar la información logística de manera que el motor PowerSync pueda filtrar un subconjunto exacto de datos pertinente exclusivamente a cada conductor, conceptualizado como un "bucket" de sincronización.²

La tabla de inventario vehicular (cargas diarias) representa este punto de anclaje. Es la representación digital del inventario físico trasladado desde la panificadora a la caja de la furgoneta al inicio de la jornada.⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria.
driver_id	UUID	No nulo. Llave foránea hacia la tabla de repartidores. Índice primario para

		partición de sincronización.
product_id	UUID	No nulo. Llave foránea hacia el catálogo de productos.
date_loaded	DATE	No nulo, defecto la fecha actual del huso horario local.
initial_quantity	NUMERIC(10,2)	No nulo. La mercadería exacta escaneada y autorizada por la administración al alba.
current_quantity	NUMERIC(10,2)	No nulo. Reflejo del stock restante tras mutaciones (ventas o devoluciones).

Esta estructura soporta operativamente el Paso 2 (Venta o Despacho) de la aplicación móvil.⁴ Durante el reparto sin conexión, el motor de SQLite local consulta su réplica de esta tabla. Si la carga del día lunes para el conductor asigna 45 kilogramos de Pan Francés y 20 docenas de Facturas Surtidas ⁴, la interfaz móvil mostrará estos topes. Al utilizar los botones masivos de incremento (+) y decremento (-), el software descuenta instantáneamente de la columna current_quantity local para evitar la sobreventa de mercadería inexistente, una invariante técnica imposible de sostener en sistemas offline sin este diseño relacional cruzado.⁴

Estructura de Clientes, Inteligencia Financiera y Rutas

La gestión de cuentas comerciales determina la complejidad de los flujos de cobro en ruta. La tabla de clientes centraliza la ficha integral de los comercios e incorpora las métricas financieras que alimentan los sistemas de alerta temprana visual en la aplicación del repartidor.⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria.
business_name	VARCHAR(150)	No nulo. Indexado mediante índices GIN con la

		extensión pg_trgm para búsquedas difusas ultrarrápidas.
phone_email	JSONB	Opcional. Empaqueta correos y teléfonos internacionales normalizados.
price_category	CHAR(1)	No nulo. Restricción CHECK (price_category IN ('A', 'B')). Define el cruce automático de precios.
address	TEXT	Opcional. Destino del reparto logístico.
current_debt	NUMERIC(12,2)	No nulo, defecto 0. Eje de la gestión de cobros y refinanciación.
allow_credit	BOOLEAN	No nulo, defecto false. Parámetro condicional que renderiza el botón "A Cta. Corriente".
fixed_order	JSONB	Opcional. Estructura de documento que define requerimientos periódicos estáticos.

El diseño del campo business_name auxiliado por índices de trigramas (pg_trgm) resulta vital. Cuando el repartidor ingresa texto en el "Buscador Inteligente", la base de datos debe ser capaz de relacionar fragmentos incompletos o con errores tipográficos con el comercio correspondiente en latencias inferiores a 50 milisegundos, ya sea en el servidor central o en el motor SQLite local.⁴

El campo current_debt sostiene la inteligencia financiera de la interfaz. Su valor (positivo para deudas acumuladas, negativo para saldos a favor) es analizado matemáticamente en tiempo de ejecución por el dispositivo móvil para renderizar etiquetas de colores (rojo para riesgo

crediticio, verde para saldos positivos) en la lista de paradas.⁴ La tabla inicial del sistema modela a la perfección esta dualidad: entidades educativas como "Colegio Nacional" presentan deudas autorizadas considerables (\$25,000) bajo una categoría de precio B, mientras que minoristas como "Kiosco El Paso" operan con saldo neto nulo bajo la categoría A, deshabilitados para generar cuenta corriente (allow_credit: false).⁴ Además, existe una entidad virtual "Consumidor Final" (ID: 999) con venta en local que actúa como sumidero para transacciones minoristas directas.⁴

La orquestación de las visitas diarias se modela mediante una tabla de enrutamiento que cruza a los conductores con sus carteras de clientes.⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria.
driver_id	UUID	No nulo. Llave foránea hacia drivers.
client_id	UUID	No nulo. Llave foránea hacia clients.
day_of_week	INTEGER	No nulo. CHECK (day_of_week BETWEEN 1 AND 7). (1=Lunes, 7=Domingo).
route_order	INTEGER	No nulo, defecto 0. Valor escalar para el ordenamiento posicional.

Esta estructura permite a la aplicación móvil consultar WHERE day_of_week = extract(isodow from current_date) y ordenar por route_order ASC, generando instantáneamente la lista lógica de las 20 a 30 paradas que el operario debe realizar, mitigando los tiempos de planificación matutina.⁴ La programación semanal revela densidades de ruta sostenidas de Lunes a Viernes, descensos operativos el Sábado y nula actividad de reparto dominguera.⁴

Motor Transaccional: Arquitectura de Ventas, Devoluciones y Erogaciones

El núcleo de alta concurrencia del sistema es la arquitectura de registro transaccional. Debe poseer la resiliencia para ingerir aluviones de registros provenientes de clientes previamente

desconectados que, de súbito, recuperan cobertura de red y sincronizan colas de operaciones masivas.²

La terminal del repartidor procesa el flujo en tres pasos secuenciales: Registro de Devoluciones (cálculo de merma), Registro de Venta (despacho real) y Gestión de Totales (cálculo final y generación de comprobante).⁴ Este flujo multidimensional se destila en un diseño de base de datos de encabezado-detalle que asegura el cumplimiento estricto de la primera forma normal relacional, facilitando la auditoría de mermas y ventas por parte de la administración.⁴

Encabezado Transaccional (Tabla sales)

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria. Generado invariablemente en el cliente móvil para certificar idempotencia.
client_id	UUID	No nulo. Llave foránea a clients(id).
driver_id	UUID	No nulo. Llave foránea a drivers(id).
transaction_date	TIMESTAMPTZ	No nulo. Estampa el momento histórico preciso de la emisión del ticket, incluso offline. ⁴
subtotal_sales	NUMERIC(12,2)	No nulo. Sumatoria bruta del valor nominal de la mercadería entregada al comercio.
total_returns	NUMERIC(12,2)	No nulo. Sumatoria de los productos mermados entregados al repartidor. ⁴
applied_debt	NUMERIC(12,2)	No nulo. Porción de la deuda histórica (Paso 3)

		sumada intencionalmente a la boleta.
final_total	NUMERIC(12,2)	No nulo. Invariante matemática estricta: <i>subtotal – devoluciones</i> .
payment_cash	NUMERIC(12,2)	No nulo, defecto 0. Fracción del total liquidada en papel moneda.
payment_transfer	NUMERIC(12,2)	No nulo, defecto 0. Fracción liquidada vía comprobantes digitales bancarios.
payment_account	NUMERIC(12,2)	No nulo, defecto 0. Valor transferido a la cuenta corriente del cliente (ticket cerrado en \$0 real). ⁴

La rigidez de este esquema previene desajustes contables. Por ejemplo, en una transacción registrada a las 08:15 AM en "Despensa Los Amigos", el sistema documenta ventas por \$42,500 y devoluciones por \$3,000. Al no existir deuda previa aplicada, el total exigible matemáticamente desciende a \$39,500, los cuales son registrados íntegramente en la columna payment_cash.⁴ La suma algebraica de los tres campos de pago debe igualar rigurosamente el final_total, garantizando la contabilidad por partida doble.

Detalles de Operación (Tabla sale_items)

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria. Generado en la terminal SQLite.
sale_id	UUID	No nulo. Llave foránea a sales(id) ON DELETE CASCADE.

product_id	UUID	No nulo. Llave foránea a products(id).
operation_type	VARCHAR(10)	No nulo. Restricción CHECK (operation_type IN ('sale', 'return')).
quantity	NUMERIC(10,2)	No nulo. Restricción CHECK (quantity > 0). Unidades físicas transaccionadas.
unit_price	NUMERIC(10,2)	No nulo. Precio histórico congelado, inmune a futuros cambios en el catálogo.

La inclusión de la columna operation_type permite condensar el Paso 1 (Devoluciones, color visual rojo) y el Paso 2 (Venta, color visual estándar) en la misma estructura tabular.⁴ Esto optimiza las operaciones de escritura masiva (Bulk Inserts) y simplifica el cruce automático post-emplasado, donde el administrador web puede ejecutar un reporte que agrupe por producto, sumando las cantidades donde el tipo sea 'sale' y restando donde sea 'return', obteniendo el rendimiento neto exacto de la materia prima.⁴

Erogaciones y Rendimiento Neto Operativo

Para calcular la rentabilidad real de cada ruta, el sistema integra una tabla auxiliar de control de gastos (expenses). Esta entidad captura las salidas de dinero justificadas durante la ruta (peajes, combustibles) o en administración central (materia prima).⁴

Columna	Tipo de Dato PostgreSQL	Restricciones y Detalles Técnicos Fundamentales
id	UUID	Llave primaria.
category	VARCHAR(100)	No nulo. Categorización analítica. Ej: 'Combustible', 'Insumos y Materia Prima'. ⁴
amount	NUMERIC(12,2)	No nulo. Restricción CHECK

		(amount > 0). Monto de la erogación.
description	TEXT	No nulo. Detalle cualitativo del evento.
origin	VARCHAR(100)	No nulo. Creador del registro (Ej. 'Roberto Sánchez' o 'Administración'). ⁴
payment_method	VARCHAR(50)	No nulo. Restricción CHECK (payment_method IN ('efectivo', 'transferencia')).
expense_date	TIMESTAMPTZ	No nulo, defecto now().

El historial operativo indica registros base como la carga de gasoil para la camioneta por \$15,000 en efectivo, y compras corporativas de harina por \$45,000 canalizadas vía transferencia bancaria.⁴ Estos datos, renderizados en los gráficos del Dashboard administrativo, permiten contrastar los ingresos por ventas directamente contra las fugas de capital diarias.⁴

Diseño de la Capa de Interfaz de Programación de Aplicaciones (API)

En el paradigma de Supabase, la API RESTful y la capa GraphQL son generadas dinámicamente y expuestas mediante PostgREST, reflejando íntimamente el esquema de base de datos subyacente.²⁰ Por tanto, la construcción de la API es sinónimo del diseño meticuloso de vistas, restricciones de esquema y procedimientos almacenados (RPC).

La delegación de la lógica de negocio al cliente móvil constituye un antipatrón arquitectónico de alto riesgo. Si el cliente procesa una venta y envía tres peticiones HTTP separadas (una para insertar la venta, otra para descontar el inventario local, y otra para aumentar la deuda del cliente), una interrupción en la red durante la secuencia generaría bases de datos rotas e inconsistentes, cobrando deudas sin registrar los despachos de mercadería. Las limitaciones inherentes a los Tipos de Datos Replicados Libres de Conflictos (CRDTs), que luchan por imponer invariantes globales estrictos (como evitar transferencias truncadas o stocks negativos), exigen que estas operaciones multidimensionales recaigan sobre los bloqueos transaccionales del servidor centralizado.⁵

Para blindar la plataforma, la API debe exponer un Procedimiento Almacenado Remoto (RPC) transaccional. En Supabase, esto se materializa como una función PostgreSQL en el lenguaje PL/pgSQL expuesta a la capa web.²³

Orquestación Transaccional: El Endpoint `rpc/process_offline_sale`

Se definirá la función `process_offline_sale` capaz de ingerir un bloque JSON (payload) que encapsula la totalidad del ticket generado en el dispositivo móvil.⁴ La ejecución transcurre bajo un bloque `BEGIN... COMMIT` atómico. Si una sola línea de código falla, el motor ejecuta un `ROLLBACK` automático, garantizando la pureza del registro histórico.

La lógica interna secuencial del RPC dictamina:

1. **Validación de Idempotencia:** El sistema verifica si el id (UUID generado en calle) de la venta ya existe en la tabla `sales`. Si es así, la función retorna silencio absoluto sin duplicar datos, protegiendo al ecosistema contra reintentos de red automáticos.
2. **Inserción Primaria:** Los datos del encabezado se insertan en la tabla matriz.
3. **Expansión de Detalles:** Se extrae el bloque de ítems del objeto JSON y se ejecutan inserciones en bloque (Bulk Inserts) en la tabla secundaria `sale_items`.
4. **Actualización Financiera Centralizada:** Si el campo `payment_account` del JSON enviado indica un valor mayor a cero, el servidor ejecuta una operación `UPDATE` sobre el registro del cliente en la tabla `clients`, incrementando dinámicamente la columna `current_debt`.⁴
5. **Reconciliación de Inventario Central:** Para que las métricas de rendimiento por furgoneta sean precisas en el panel administrativo, el servidor actualiza la tabla de cargas diarias (loads), sumando los retornos y restando los despachos confirmados, facilitando al sistema auditar posibles pérdidas o robos de materia prima.

Este diseño centralizado elimina la necesidad de algoritmos de resolución de conflictos deterministas en la terminal del trabajador, asegurando que el estado de la cuenta corriente converja de forma lineal y auditada en el núcleo de la empresa.

Arquitectura de Seguridad: Row Level Security (RLS)

El acceso distribuido a los datos logísticos requiere un blindaje total. Permitir que dispositivos SQLite en entornos públicos posean acceso no filtrado o que la API autogenerada reciba peticiones anónimas compromete el cumplimiento de las normativas de privacidad comerciales. Las políticas RLS de PostgreSQL restringen operativamente los datos fila por fila en función del token de autorización proporcionado.¹⁸

El ecosistema posee dos entidades principales: Administradores (acceso global a métricas, configuraciones y finanzas) y Repartidores (operarios confinados a la geografía de su ruta).⁴

Optimización de Invocaciones RLS para Alto Rendimiento

Una deficiencia documentada en el motor de Supabase surge al evaluar políticas RLS que contienen subconsultas u operaciones de reunión (JOIN) contra otras tablas. Dado que el motor evalúa la política de forma iterativa por cada fila escaneada en un comando `SELECT`, un listado de miles de transacciones puede multiplicar exponencialmente el tiempo de respuesta, colapsando el servidor y degradando severamente el dashboard.²⁵

Para erradicar este cuello de botella, la arquitectura adopta un patrón basado en "Custom Claims" (reclamos personalizados) inyectados directamente en el JSON Web Token (JWT)

durante el flujo de inicio de sesión del empleado.²⁷ Utilizando la función auxiliar optimizada `auth.jwt()`, el sistema puede verificar jerarquías sin golpear los discos magnéticos.¹⁸ Un reclamo como `{"role": "admin"}` se incrusta en el espacio inmutable `raw_app_meta_data`, impidiendo que los usuarios maliciosos intercepten y escalen privilegios localmente.¹⁸

Reglas RLS por Dominio

1. Catálogo (products) y Cartera de Clientes (clients) Dado que los algoritmos de búsqueda móvil precisan consultar la totalidad del catálogo universal para identificar ítems a devolver, e inspeccionar comercios para aplicar precios de Categoría A o B, los permisos de lectura deben ser expeditivos.⁴

- **Lectura (SELECT):** Permisiva universalmente para perfiles logueados, validando `auth.role() = 'authenticated'`.
- **Modificación (INSERT, UPDATE, DELETE):** Confinada axiomáticamente a credenciales administrativas, verificada mediante `auth.jwt() -> 'app_metadata' -> 'role' = 'admin'`.

2. Transacciones Sensibles (sales, sale_items, expenses)

El aislamiento corporativo requiere que los repartidores carezcan de facultades para auditar el volumen de recaudación de sus compañeros de flota.

- **Lectura (SELECT):** Transparente para administradores. Para los conductores, el sistema valida que el identificador del usuario solicitante concuerde matemáticamente con el `driver_id` del registro (Ej. `driver_id = auth.uid()`).
- **Escritura (INSERT):** Facultada si el `driver_id` del nuevo registro coincide imperativamente con la firma criptográfica del emisor del JWT. Modificaciones retrospectivas (UPDATE) se bloquean a nivel de base de datos para todas las entidades transaccionales, forzando la creación de notas de crédito compensatorias, un estándar contable infranqueable.

3. Privilegios Elevados Controlados (SECURITY DEFINER) Un riesgo sistémico inherente surge cuando la aplicación móvil requiere alterar la tabla de clientes para asentar deudas originadas en ventas financiadas.⁴ Si se conceden permisos explícitos de UPDATE sobre la tabla `clients` a los conductores de furgonetas, se abre un vector de ataque que permitiría alterar categorías de descuento o correos electrónicos por fuera del diseño aplicativo. Para resolver la dicotomía, la tabla `clients` negará rotunda y universalmente los privilegios UPDATE a cualquier rol que no sea administrador. El puente tecnológico consiste en definir la función RPC (`process_offline_sale`) declarando el modificador `SECURITY DEFINER`.²⁰ A diferencia del estándar `SECURITY INVOKER` que acata las políticas del ejecutor subyacente¹⁸, el `SECURITY DEFINER` indica al motor de PostgreSQL que, temporalmente, los comandos se ejecutarán bajo la sombrilla de privilegios supremos del arquitecto que diseñó el código.²³ Esto faculta a la aplicación a escalar el saldo de la deuda de manera aséptica, paramétrica y auditable, imposibilitando inyecciones indeseadas y clausurando el perímetro de seguridad.

Topología del Middleware de Sincronización (PowerSync)

La orquestación del esquema Offline-First culmina en la interconexión con el servicio de PowerSync. La grandeza de este middleware estriba en su naturaleza etérea: no demanda

mutaciones invasivas en la base de datos principal, basando su funcionamiento en las capacidades nativas de replicación lógica de PostgreSQL (Change Data Capture - CDC).¹ A nivel infraestructura, se emite el comando de habilitación de publicación `CREATE PUBLICATION powersync FOR ALL TABLES`, acoplado a la gestión de un rol de servicio asilado (`powersync_role`) portador de concesiones exclusivas de lectura global (`SELECT`) e inmunidad a las reglas RLS (`BYPASSRLS`).²⁸ Esta cimentación transfiere al servicio satélite un flujo incesante de alteraciones de estado. Sin embargo, el refinamiento arquitectónico emerge en el estrato de las Reglas de Sincronización (Sync Streams) dictaminadas mediante sintaxis SQL transpuesta en documentos YAML.²⁹

Particionado Lógico (Buckets)

Los "buckets" definen qué fragmentos específicos de la base de datos central merecen habitar el almacenamiento SQLite del dispositivo perimetral.¹⁹ El sistema declara los siguientes canales de transmisión:

1. **Estrato Base Estático (Global):** Transmisión unidireccional permanente de la tabla `products`. Las fluctuaciones de precios unitarios o nuevas altas impactan instantáneamente en las terminales activas. Se excluye algorítmicamente la columna `bakery_stock` del bucket para aligerar la carga de red y ocultar información corporativa innecesaria al repartidor de campo.
2. **Estrato de Enrutamiento Geográfico:** Despliegue focalizado de la tabla `clients` y `weekly_routes`. La regla filtra los comercios que posean un enlace relacional activo con el identificador del conductor evaluado en la macro de sesión, canalizando los datos para la representación visual en la lista ordenada de la aplicación móvil.⁴
3. **Estrato Dinámico de Inventario en Tránsito:** Un cruce relacional que anida el `driver_id` con la tabla de `loads`. Este canal asegura que la memoria local posea consciencia inmediata de las dotaciones de Pan de Miga o Bizcochitos físicamente presentes en la unidad automotriz, habilitando los cálculos de decremento del Paso 2 sin depender de microservicios remotos.⁴
4. **Estrato Transaccional Autocontenido:** Filtrado de registros de ventas de las últimas 72 horas pertenecientes al conductor. Facilita la carga de reportes diarios, emisión de tickets reimpresos por Bluetooth, y conciliación visual de las ventas pagadas en efectivo versus las canalizadas digitalmente, sin consumir ciclos de CPU central.⁴

Consenso y Convergencia de Datos

El diseño previene colisiones catastróficas. Al converger eventos concurrentes (por ejemplo, el conductor cobra una boleta vencida en la calle y simultáneamente el gerente aprueba una refinanciación en el panel administrativo), las arquitecturas naive experimentan pérdidas de intención (Last Write Wins) o comportamientos erráticos en la interfaz.⁵

El flujo unidireccional ascendente de PowerSync consolida los comandos locales sobre el checkpoint verificado, almacenándolos en una cola inmutable.² Al enlazarse con el servidor, la lógica transaccional incrustada en el RPC de PostgreSQL asimila y despliega las actualizaciones numéricas matemáticamente relativas (Ej. Deuda Actual = Deuda Actual + \$Nuevo Valor) en

lugar de imposiciones de valores absolutos.² El servidor funge como juez inapelable del ordenamiento lógico; si el estado global dictamina que un cliente superó un límite de crédito bloqueante, la operación se revierte en la central, y el fallo se sincroniza descendentemente para alertar a la interfaz del usuario.² Esta delegación sistemática asegura la armonía, descargando al hardware móvil de arbitrajes distributivos costosos y consolidando un ecosistema transaccional rápido, incorruptible y blindado contra el caos de la intermitencia celular.

Fuentes citadas

1. PowerSync | Works With Supabase, acceso: junio 3, 2026, <https://supabase.com/partners/integrations/powersync>
2. Introducing PowerSync v1.0: Postgres<>SQLite Sync Layer, acceso: junio 3, 2026, <https://powersync.com/blog/introducing-powersync-v1-0-postgres-sqlite-sync-layer>
3. PowerSync: Backend DB - SQLite sync engine | For Postgres, MongoDB, MySQL, SQL Server, acceso: junio 3, 2026, <https://powersync.com/>
4. panificadoraapp-etapa-dos.tsx
5. Blog | Dmytro Zharii, acceso: junio 3, 2026, <https://blog.zharii.com/blog>
6. Building an Offline-First PWA Notes App with Next.js, IndexedDB, and Supabase - Israel, acceso: junio 3, 2026, <https://oluwadaprof.medium.com/building-an-offline-first-pwa-notes-app-with-next-js-indexeddb-and-supabase-f861aa3a06f9>
7. Proof of concept to explore timezones in PostgreSQL and between server/browser - GitHub Gist, acceso: junio 3, 2026, <https://gist.github.com/06fe0872ce9e8dfb0d522aa2dedfc5c2>
8. How to Convert UTC to Local Time Zone in PostgreSQL - PopSQL, acceso: junio 3, 2026, <https://popsql.com/learn-sql/postgresql/how-to-convert-utc-to-local-time-zone-in-postgresql>
9. Documentation: 8.1: Date/Time Key Words - PostgreSQL, acceso: junio 3, 2026, <https://www.postgresql.org/docs/8.1/datetime-keywords.html>
10. How to configure postgres Timezone? - PitStop ManageEngine, acceso: junio 3, 2026, <https://pitstop.manageengine.com/portal/en/kb/articles/how-to-configure-postgres-timezone>
11. Incorrect Argentina time zone? : r/PostgreSQL - Reddit, acceso: junio 3, 2026, https://www.reddit.com/r/PostgreSQL/comments/1khhrql/incorrect_argentina_time_zone/
12. Wrong PostgreSQL timezone name: America/Buenos_Aires is invalid #39187 - GitHub, acceso: junio 3, 2026, <https://github.com/dbeaver/dbeaver/issues/39187>
13. How to get the list of timezones supported by PostgreSQL? - Stack Overflow, acceso: junio 3, 2026, <https://stackoverflow.com/questions/54000372/how-to-get-the-list-of-timezones-supported-by-postgresql>

14. meridian-react-native 0.4.0 on npm - Libraries.io, acceso: junio 3, 2026, <https://libraries.io/npm/meridian-react-native>
15. GitHub - prabhask5/stellar: A self-hosted, offline-first productivity PWA with goal tracking, daily tasks, routines, Pomodoro focus timer, and project organization. Built with SvelteKit, Dexie.js, and Supabase with real-time multi-device sync., acceso: junio 3, 2026, <https://github.com/prabhask5/stellar>
16. USPTO Patent Portfolio — 16 Filings - PasskeyBridge, acceso: junio 3, 2026, <https://passkeybridge.io/patents>
17. Built with Opus 4.7: a Claude Code hackathon - Cerebral Valley, acceso: junio 3, 2026, <https://cerebralvalley.ai/e/built-with-4-7-hackathon/hackathon/gallery>
18. Row Level Security | Supabase Docs, acceso: junio 3, 2026, <https://supabase.com/docs/guides/database/postgres/row-level-security>
19. Offline-First Apps Made Simple: Supabase + PowerSync, acceso: junio 3, 2026, <https://powersync.com/blog/offline-first-apps-made-simple-supabase-powersync>
20. Authorization via Row Level Security | Supabase Features, acceso: junio 3, 2026, <https://supabase.com/features/row-level-security>
21. Securing your API | Supabase Docs, acceso: junio 3, 2026, <https://supabase.com/docs/guides/api/securing-your-api>
22. ElectricSQL (Legacy) Vs PowerSync, acceso: junio 3, 2026, <https://powersync.com/blog/electricsql-vs-powersync>
23. RLS On Views? : r/Supabase - Reddit, acceso: junio 3, 2026, https://www.reddit.com/r/Supabase/comments/1mt4imr/rls_on_views/
24. Lock Down Your Data: Implement Row-Level Security Policies in Supabase SQL, acceso: junio 3, 2026, <https://dev.to/thebenforce/lock-down-your-data-implement-row-level-security-policies-in-supabase-sql-4p82>
25. Performance and Security Advisors | Supabase Docs, acceso: junio 3, 2026, https://supabase.com/docs/guides/database/database-advisors?int=0006_multiple_permissive_policies
26. Troubleshooting | RLS Performance and Best Practices - Supabase Docs, acceso: junio 3, 2026, <https://supabase.com/docs/guides/troubleshooting/rls-performance-and-best-practices-Z5Jjwv>
27. Row Level Security (RLS): Performance implications : r/Supabase - Reddit, acceso: junio 3, 2026, https://www.reddit.com/r/Supabase/comments/14k49l6/row_level_security_rls_performance_implications/
28. Source Database Setup - PowerSync Docs, acceso: junio 3, 2026, <https://docs.powersync.com/configuration/source-db/setup>
29. Supabase - PowerSync Docs, acceso: junio 3, 2026, <https://docs.powersync.com/integrations/supabase/guide>
30. Self-Hosted Instance Configuration - PowerSync Docs, acceso: junio 3, 2026, <https://docs.powersync.com/configuration/powersync-service/self-hosted-instances>

31. PowerSync and Supabase: Just the Basics, acceso: junio 3, 2026,
<https://powersync.com/blog/powersync-and-supabase-just-the-basics>